

## **Лабораторная работа №7**

**Заболотнов Николай Михайлович**

**6204-010302D**

## Task 1

Сделаем так, чтобы все объекты типа TabulatedFunction можно было использовать в цикле for-each. Пусть интерфейс TabulatedFunction наследуется от Iterable<FunctionPoint>. В классах ArrayTabulatedFunction и LinkedListTabulatedFunction добавим метод iterator() (рис 1 и 2). В методе main класса Main проверим работу итераторов класса (рис 3).

```
public Iterator<FunctionPoint> iterator() {  
    return new Iterator<FunctionPoint>() {  
        private int index = 0;  
  
        public boolean hasNext() {  
            return index < PointsCount;  
        }  
  
        public FunctionPoint next() {  
            if (hasNext())  
                return new FunctionPoint(points[index++]);  
            throw new NoSuchElementException("Нет больше точек");  
        }  
  
        public void remove() {  
            throw new UnsupportedOperationException("Нельзя удалить элемент");  
        }  
    };  
}
```

Рис 1

```

public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private FunctionNode node = head;
        private int index = 0;

        public boolean hasNext() {
            return index < size;
        }

        public FunctionPoint next() {
            if (hasNext()) {
                FunctionPoint point = new FunctionPoint(node.point);
                ++index;
                node = node.next;
                return point;
            }
            throw new NoSuchElementException("Нет больше точек");
        }

        public void remove() {
            throw new UnsupportedOperationException("Нельзя удалить элемент");
        }
    };
}

```

Рис 2

```

TabulatedFunction f1 = TabulatedFunctions.tabulate(new Exp(), 0, 10, 21);
for (FunctionPoint p : f1) {
    System.out.println(p);
}

```

Рис 3

**Вывод 1:**

```
(0.0,1.0)
(0.5,1.6487212707001282)
(1.0,2.718281828459045)
(1.5,4.4816890703380645)
(2.0,7.38905609893065)
(2.5,12.182493960703473)
(3.0,20.085536923187668)
(3.5,33.11545195869231)
(4.0,54.598150033144236)
(4.5,90.01713130052181)
(5.0,148.4131591025766)
(5.5,244.69193226422038)
(6.0,403.4287934927351)
(6.5,665.1416330443618)
(7.0,1096.6331584284585)
(7.5,1808.0424144560632)
(8.0,2980.9579870417283)
(8.5,4914.768840299134)
(9.0,8103.083927575384)
(9.5,13359.726829661873)
(10.0,22026.465794806718)
```

Рис 4

Задание выполнено.

## Task 2

В пакете `functions` напомним интерфейс фабрик табулированных функций `TabulatedFunctionFactory`, содержащий перегруженные методы `createTabulatedFunction()`. Добавим реализацию этих методов в классы `ArrayTabulatedFunctionFactory` и `LinkedListTabulatedFunctionFactory` (рис 5 и 6). В классе `TabulatedFunctions` объявим приватное статическое поле `TabulatedFunctionFactory`, метод `setTabulatedFunctionFactory()` (рис 7), заменяющий объект фабрики, и три перегруженных метода `createTabulatedFunction()` (рис 8), создающих объекты табулированных функций. Проверим работу в классе `Main` (рис 9).

```

public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory {
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }

    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new ArrayTabulatedFunction(points);
    }
}

```

Рис 5

```

public static class LinkedListTabulatedFunctionFactory implements TabulatedFunctionFactory {
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new LinkedListTabulatedFunction(leftX, rightX, pointsCount);
    }

    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new LinkedListTabulatedFunction(leftX, rightX, values);
    }

    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new LinkedListTabulatedFunction(points);
    }
}

```

Рис 6

```

public static void setTabulatedFunctionFactory(TabulatedFunctionFactory factory) {
    TabulatedFunctions.factory = factory;
}

```

Рис 7

```

public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
    return TabulatedFunctions.factory.createTabulatedFunction(leftX, rightX, pointsCount);
}

public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
    return TabulatedFunctions.factory.createTabulatedFunction(leftX, rightX, values);
}

public static TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
    return TabulatedFunctions.factory.createTabulatedFunction(points);
}

```

Рис 8



```

Function f2 = new Cos();
TabulatedFunction tf;
tf = TabulatedFunctions.tabulate(f2, 0, Math.PI, 11);
System.out.println(tf.getClass());
TabulatedFunctions.setTabulatedFunctionFactory(new LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory());
tf = TabulatedFunctions.tabulate(f2, 0, Math.PI, 11);
System.out.println(tf.getClass());
TabulatedFunctions.setTabulatedFunctionFactory(new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory());
tf = TabulatedFunctions.tabulate(f2, 0, Math.PI, 11);
System.out.println(tf.getClass());

```

Рис 9

## Вывод 2:

```

class functions.ArrayTabulatedFunction
class functions.LinkedListTabulatedFunction
class functions.ArrayTabulatedFunction

```

Рис 10

Задание выполнено.

## Task 3

В классе TabulatedFunctions добавим три перегруженных версии метода createTabulatedFunction(), которые будут принимать ссылку на описание класса и параметры конструктора (рис 11). Перегрузим методы, создающие объекты табулированных функций, добавив к принимающим значениям ссылки типа Class. Проверим работу в классе Main (рис 12).

```

public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> functionClass, double leftX, double rightX, int pointsCount) {
    try {
        return functionClass.getConstructor(double.class, double.class, int.class).newInstance(leftX, rightX, pointsCount);
    } catch (Exception e) {
        throw new IllegalArgumentException(e);
    }
}

public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> functionClass, double leftX, double rightX, double[] values) {
    try {
        return functionClass.getConstructor(double.class, double.class, double[].class).newInstance(leftX, rightX, values);
    } catch (Exception e) {
        throw new IllegalArgumentException(e);
    }
}

public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> functionClass, FunctionPoint[] points) {
    try {
        return functionClass.getConstructor(FunctionPoint[].class).newInstance((Object) points);
    } catch (Exception e) {
        throw new IllegalArgumentException(e);
    }
}

```

Рис 11

```

TabulatedFunction f3;
f3 = TabulatedFunctions.createTabulatedFunction(ArrayTabulatedFunction.class, 0, 10, 3);
System.out.println(f3.getClass());
System.out.println(f3);
f3 = TabulatedFunctions.createTabulatedFunction(ArrayTabulatedFunction.class, 0, 10, new double[] {0, 10});
System.out.println(f3.getClass());
System.out.println(f3);
f3 = TabulatedFunctions.createTabulatedFunction(LinkedListTabulatedFunction.class, new FunctionPoint[] {new FunctionPoint(0, 0), new FunctionPoint(10, 10)});
System.out.println(f3.getClass());
System.out.println(f3);
f3 = TabulatedFunctions.tabulate(LinkedListTabulatedFunction.class, new Sin(), 0, Math.PI, 11);
System.out.println(f3.getClass());
System.out.println(f3);

```

Рис 12

## Вывод 3:

```

class functions.ArrayTabulatedFunction
{(0.0,0.0), (5.0,0.0), (10.0,0.0)}
class functions.ArrayTabulatedFunction
class functions.ArrayTabulatedFunction
{(0.0,0.0), (10.0,10.0)}
class functions.LinkedListTabulatedFunction
{(0.0,0.0), (10.0,10.0)}
class functions.LinkedListTabulatedFunction
{(0.0,0.0), (0.3141592653589793,0.3090169943749474), (0.6283185307179586,0.5877852522924731), (0.94247
77960769379,0.8090169943749475), (1.2566370614359172,0.9510565162951535), (1.5707963267948966,1.0), (1
.8849555921538759,0.9510565162951536), (2.199114857512855,0.8090169943749475), (2.5132741228718345,0.5
877852522924732), (2.827433388230814,0.3090169943749475), (3.141592653589793,1.2246467991473532E-16)}

```

Рис 13

Задание выполнено.